Lecture 3
Solutions

Homework 3: Rescue the repository

Solutions to Homework 3

```
$ bash codeHints/L03/setup_hw3.sh
$ cd practice/hw3
$ git log --oneline
e672c50 Update notse
6e1cc4f Buy fireworks
809893e Add recipe notes
2217470 Add shopping list
$ git restore list.txt
$ git restore --staged notes.txt
$ git commit --amend -m "Update notes"
$ git revert --no-edit 6e1cc4f
$ git status --short
 M notes.txt
$ printf 'fireworks=6e1cc4f\ncommits=4\nno-undo=restore\n' > ../hw3_answers.txt
```

(Your IDs for the first three commits match these because the setup script fixes author and dates. The amended and revert commits get new IDs.)

3.1. `git log --oneline` shows the ID `6e1cc4f` next to `Buy fireworks` and four lines, so `commits=4`. The answer file lives in `practice/`, outside the repository, so it does not show up as an untracked file.

3.2. `git diff list.txt` first shows the junk line `+oops delete me`. `git restore list.txt` removes it. This is the destructive command: it is correct here, because the line really was junk. You must do this *before* the revert, because the revert changes `list.txt` too and Git will not overwrite your uncommitted edit.

3.3. `git restore --staged notes.txt` moves the `TODO: add syrup` line from "staged" back to "modified". The line stays in the file. Check with `git status --short`: the first column is now empty.

3.4. `git commit --amend -m "Update notes"` rewrites the last commit with the corrected message. The order matters: if you amend while `notes.txt` is still staged, the TODO line is pulled into the commit. That is why 3.3 comes first. If you made that mistake, run `setup_hw3.sh --reset` and start again.

3.5. `git revert --no-edit 6e1cc4f` creates `Revert "Buy fireworks"` on top of the amended commit. The history has five commits now. It works without conflict because the commits after `Buy fireworks` only touched `notes.txt`.

3.6. The answer is `restore`. It discards uncommitted work for good. (`restore --staged` keeps your edit; `amend` leaves the old commit abandoned but recoverable, as Lecture 10 shows; and `revert` just adds a commit. None of the three deletes work from your files.)

Common mistakes.

- Doing 3.4 before 3.3: the TODO line gets committed. Reset and redo in the right order.
- Running `git revert` without `--no-edit` and getting stuck in the editor: press `Esc`, type `:wq`, Enter to accept the default message.
- Reverting the wrong commit (the typo commit instead of `Buy fireworks`). Look at the ID in `git log --oneline` before you revert.
- Using `git reset`, which this lecture did not teach: it can rewrite history in ways that this lecture does not cover, and the checker may flag the result.